

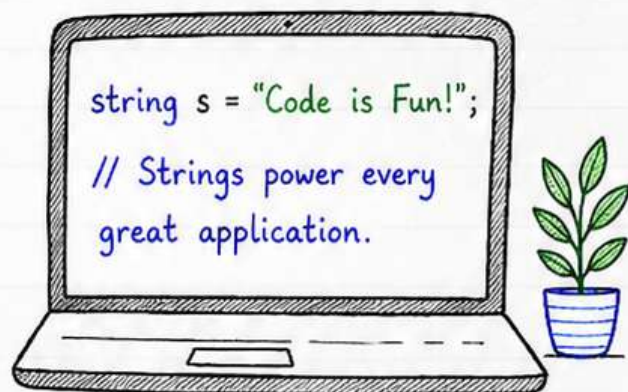
STRINGS

☆ ⇒ Master Strings for Coding Interviews ⇐ ☆

TOPICS COVERED

- ① What is a String? ?
- ② Memory Representation 
- ③ String Operations (:)
- ④ Time Complexity ⌚
- ⑤ Character Manipulation A↔a
- ⑥ Reverse String ⇌
- ⑦ Palindrome 
- ⑧ Anagrams (A) (B)
- ⑨ Frequency Count 
- ⑩ Most Asked Interview Questions ☆

"Strings are the building blocks of text and the foundation of many real-world solutions."



</>



REMEMBER

Strong foundation in Strings → Stronger problem solving!



Learn



Practice



Code



Excel



⇒ 2. WHAT IS A STRING? ⇐

DEFINITION :

A **String** is a sequence of characters stored together to represent text.

String =
Seq. of
characters

EXAMPLES :

```
string name = "Ranjan";  
string city = "Delhi";
```

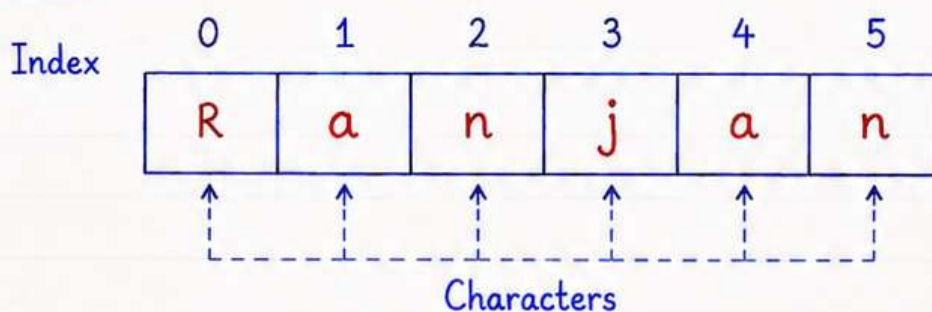


CHARACTERISTICS :

- ☆ Collection of characters.
- ☆ Zero-based indexing.
- ☆ Immutable in some languages (like Java, Python).
- ☆ Mutable in C++.

Characters
combine to
form **Strings** !

DIAGRAM :



"Ranjan"
has 6
characters
(size = 6)

APPLICATIONS :

- ✓ Search  - Searching words, patterns, etc.
- ✓ Password Validation  - Checking strength of passwords.
- ✓ Parsing  - Breaking text into meaningful parts.
- ✓ Text Processing  - Editing, formatting, analyzing text.

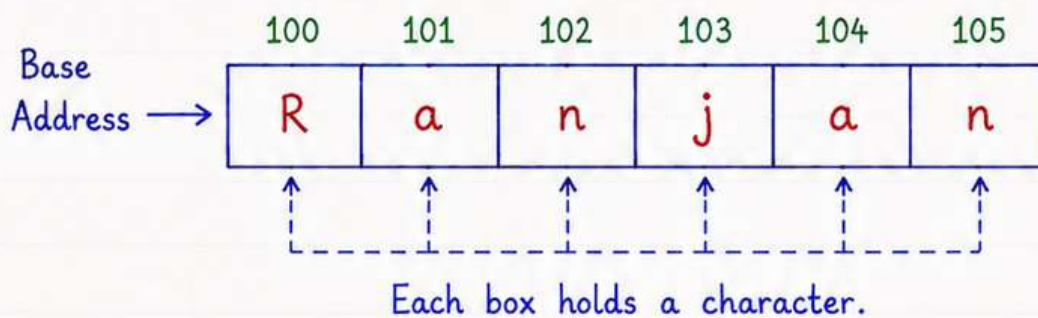
Strings
are everywhere
in coding!



3. MEMORY REPRESENTATION & TIME COMPLEXITY

MEMORY LAYOUT :

A String is stored as a sequence of characters in contiguous memory locations.



Continuous Memory
= Fast Access

STRING FUNCTIONS & TIME COMPLEXITY :

	Operation	Complexity
★	Access ($s[i]$)	$O(1)$
★	Traversal	$O(n)$
★	Compare ($s1 == s2$)	$O(n)$
★	Append (at end)	$O(1)^*$
★	Insert (at position)	$O(n)$
★	Delete (at position)	$O(n)$

* Append is $O(1)$ amortized.
(Occasional reallocation may take $O(n)$)



INTERVIEW TIP :



Characters are stored continuously in memory.
That's why strings support fast random access!

```
string s = "Ranjan";
cout << s[2]; // n
```

4. COMMON STRING OPERATIONS



Strings have many **useful built-in operations** that make our work easy and efficient. 

COMMON OPERATIONS :

Operation	Description
★ <code>length()</code>	Returns the number of characters in the string.
★ <code>size()</code>	Returns the size (same as length).
★ <code>substr(pos, len)</code>	Returns substring of length 'len' from position 'pos'.
★ <code>find(ch / str)</code>	Returns the index of first occurrence of character or substring. If not found, returns <code>npos</code> .
★ <code>erase(pos, len)</code>	Removes 'len' characters starting from position 'pos'.
★ <code>insert(pos, str)</code>	Inserts the given substring 'str' at position 'pos'.
★ <code>replace(pos, len, str)</code>	Replaces 'len' characters starting from 'pos' with new string 'str'.

EXAMPLE :

```
string s = "Hello";
cout << s.length();    // Output: 5
cout << s.size();      // Output: 5
cout << s.substr(1, 3); // Output: "ell"
                        (index 1, length 3)
```

In C++,
length() and **size()**
both return the same
value for strings.



VISUAL EXAMPLES :

1) `length() / size()`

Input: "Hello"

Output: **5**

2) `substr(1, 3)`

Input: "Hello"

0	1	2	3	4
H	e	l	l	o

Output: "ell"

3) `find("ll")`

Input: "Hello"

0	1	2	3	4
H	e	l	l	o

Output: **2**
(first index of "ll")

4) `erase(1, 2)`

Input: "Hello"

0	1	2	3	4
H	e	l	l	o

Output: "Hlo"

5) `insert(1, "i")`

Input: "Helo"

0	1	2	3
H	e	l	o

Output: "Hielo"

6) `replace(1, 2, "i")`

Input: "Hello"

0	1	2	3	4
H	e	l	l	o

Output: "Hillo"

7) `find('x')`

Input: "Hello"

Output: **npos**
(Not found)

NOTE :

`npos` = a special value returned when the substring is not found.



5. REVERSE A STRING

Reversing a string means writing the characters in the **opposite order**.



PROBLEM :

Given a string, reverse it.

EXAMPLE :



Think:

Start from end and go to start!



METHODS :

① Two Pointer Approach

Index →	0	1	2	3	4
String →	H	E	L	L	O
	L →			← R	

- ★ Place one pointer at start (L) and other at end (R).
- ★ Swap characters at L and R.
- ★ Move L forward and R backward.
- ★ Repeat until $L \geq R$.

② Using Reverse Function

Most easy approach using built-in function.

```
reverse(s.begin(), s.end());
```

Works in C++ STL

EXAMPLE CODE (C++) :

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s = "Hello";
    reverse(s.begin(), s.end());
    cout << s; // Output: olleH
    return 0;
}
```



DRY RUN (Two Pointer) :

Step	String	L	R	Action
0	H E L L O	0	4	Initial
1	O E L L H	1	3	Swap H & O
2	O L L E H	2	2	Swap E & L
3	O L L E H	3	1	Stop ($L > R$)

COMPLEXITY :

Time Complexity : $O(n)$

Space Complexity : $O(1)$



INTERVIEW TIP :

- ✓ Two pointer is great for manual control.
- ✓ Built-in reverse() is clean and less error-prone.



6. PALINDROME

DEFINITION : A string is a palindrome if it reads the same forward and backward.



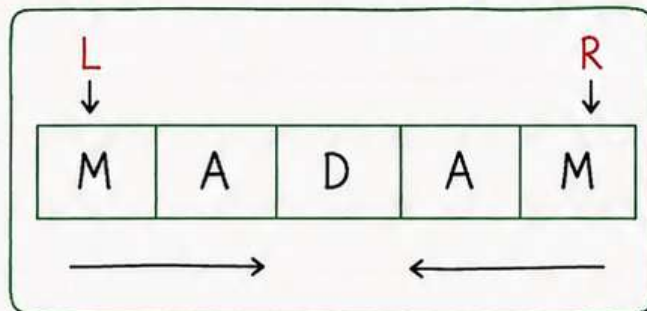
EXAMPLES :

- ★ MADAM ☒
- ★ LEVEL ☒
- ★ HELLO ☐

Palindromes are
symmetric
in nature!



APPROACH : Two Pointer Approach



EXAMPLE CODE (C++) :

```
bool isPalindrome(string s) {
    int L = 0, R = s.length() - 1;
    while (L < R) {
        if (s[L] != s[R]) return false;
        L++;
        R--;
    }
    return true;
}
```



★ STEPS :

- ① Place L at start and R at end.
- ② Compare s[L] and s[R].
- ③ If same, move L++ and R--
- ④ Repeat until L >= R.
- ⑤ If all matches → Palindrome
else → Not Palindrome

DRY RUN : "MADAM"

Step	L	R	Compare	Result
0	0	4	M vs M	Match ✓
1	1	3	A vs A	Match ✓
2	2	2	D vs D	Match ✓

L >= R → Palindrome



COMPLEXITY :

Time Complexity : $O(n)$
Space Complexity : $O(1)$

INTERVIEW TIP :

- ★ Don't forget to handle edge cases (empty string, single char).
- ★ Case sensitivity depends on the problem statement.



7. ANAGRAMS & FREQUENCY COUNT

Anagrams are two strings that contain the same characters with the same frequency, but in different order.



ANAGRAM :

Two strings are anagrams if we can rearrange characters of one string to get the other.

Example :

listen $\rightleftharpoons$ silent
(Anagram)



FREQUENCY COUNT (Example)

String : "aabac"

Frequency Array (a - z) :

Char	a	b	c	d ...	z
Freq	2	0	1	0 ...	0

From above : a $\rightarrow$ 2

b $\rightarrow$ 0 (others $\rightarrow$ 0)

c $\rightarrow$ 1



APPROACHES TO CHECK ANAGRAM :

① Sorting Approach

- ★ Sort both strings.
- ★ If sorted strings are same, then they are anagrams.

Ex :

s1 = "listen" $\rightarrow$ eilnst

s2 = "silent" $\rightarrow$ eilnst

Works but takes $O(n \log n)$



② Frequency Count Approach

- ★ Count frequency of each character in both strings.
- ★ If frequency arrays are same, strings are anagrams.

Ex :

s1 = "listen" $\rightarrow$ [1,1,1,1,1,0,...]

s2 = "silent" $\rightarrow$ [1,1,1,1,1,0,...]

Same $\rightarrow$ Anagrams



TIP :

Use frequency count when constraints are large (better than sorting).



FREQUENCY ARRAY NOTES :

- ★ For lowercase letters (a-z) $\rightarrow$ use array of size 26.
- ★ For ASCII (0-127) $\rightarrow$ use array of size 128.
- ★ For extended ASCII $\rightarrow$ size 256.
- ★ Initialize array with 0.

COMPLEXITY :

Time Complexity : $O(n)$

Space Complexity : $O(1)$



SUMMARY

- ✓ Anagrams have same characters with same frequency.
- ✓ Two main approaches : Sorting ($O(n \log n)$) and Frequency Count ($O(n)$).
- ✓ Frequency Count is better and optimal.
- ✓ Used in many interview problems!



8. CHARACTER MANIPULATION

Characters in a string can be transformed, compared and checked using **ASCII values** and **built-in functions**.

TOPICS COVERED :

- ★ Uppercase ↔ Lowercase
- ★ ASCII
- ★ Digits (0-9)
- ★ Special Characters

ASCII BASICS :

Every character has a unique **ASCII value**.

A → 65

a → 97

Difference
= 32



Character	A	B	C	...	Z	a	b	c	...	z	0	1	...	9
ASCII	65	66	67	...	90	97	98	99	...	122	48	49	...	57

UPPERCASE ↔ LOWERCASE :

To convert between cases using ASCII :

- ★ If 'a' to 'z', subtract 32 to get Uppercase.
- ★ If 'A' to 'Z', add 32 to get Lowercase.

Examples :

'a' (97) - 32 = 'A' (65)

'b' (98) - 32 = 'B' (66)

'Z' (90) + 32 = 'z' (122)

Diagram :

Uppercase (65-90)

↑ - 32

Lowercase (97-122)

DIGITS (0 - 9) :

Digits have ASCII values 48 to 57.

Ex : '0' = 48 , '1' = 49 , ... , '9' = 57

Check : `ch >= '0' && ch <= '9'`

SPECIAL CHARACTERS :

Special characters include symbols like

! @ # \$ % ^ & * () _ + - = [] { }
and many more.

We can check them using ASCII range or built-in functions.

USEFUL FUNCTIONS (<cctype> library in C++) :

Function	Purpose	Example	Returns
<code>toupper(ch)</code>	Convert lowercase to uppercase	<code>toupper('a')</code>	'A'
<code>tolower(ch)</code>	Convert uppercase to lowercase	<code>tolower('B')</code>	'b'
<code>isdigit(ch)</code>	Check if character is digit	<code>isdigit('7')</code>	true
<code>isalpha(ch)</code>	Check if character is alphabet	<code>isalpha('x')</code>	true
<code>isalnum(ch)</code>	Check if character is alphanumeric	<code>isalnum('k')</code>	true
<code>isspace(ch)</code>	Check if character is whitespace	<code>isspace(' ')</code>	true

SMALL CODE EXAMPLE :

```
char ch = 'G';
cout << toupper(ch); // G
cout << tolower(ch); // g
cout << isalpha(ch); // 1 (true)
cout << isdigit(ch); // 0 (false)
```



INTERVIEW TIP :

ASCII based questions are **very common** in interviews. Practice them well!

COMMON ASCII FACTS :

★ Total ASCII characters = 128 (0 - 127)

★ Printable characters = 32 to 126

★ Difference between 'a' and 'A' = 32



STRINGS



Page 9
of Many



MOST ASKED STRING INTERVIEW QUESTIONS

Easy

- Reverse String
- Palindrome
- Count Vowels
- Remove Spaces



Medium

- Longest Common Prefix
- Valid Anagram
- Group Anagrams
- String Compression



Hard

- Longest Substring Without Repeating Characters
- Minimum Window Substring



Company Tags



Google



Amazon



Microsoft



Adobe



Meta



STRINGS



Page 10
of Many



CHEAT SHEET

QUICK REVISION

- ✓ String Declaration
- ✓ Common Functions
- ✓ Time Complexity
- ✓ Two Pointer Pattern

- ✓ Frequency Array
- ✓ ASCII Tricks
- ✓ Interview Tips



★ 1. String Declaration

- `std::string s = "hello";`
- `char str[] = "hello";`
- `char str[10];`
- `string s(n, 'a'); // n times 'a'`

★ 2. Common Functions

- `s.length()` → length of string
- `s.size()` → same as `length()`
- `s.empty()` → check if empty
- `s.push_back(ch)` → add char at end
- `s.substr(i, j)` → substring `[i, j-1]`
- `s.find(ch)` → first occurrence
- `s.erase(i, j)` → erase `j` chars from `i`

★ 3. Time Complexity

- Access by index → $O(1)$
- Search / find → $O(n)$
- Insert / Delete → $O(n)$
- Substring → $O(k)$

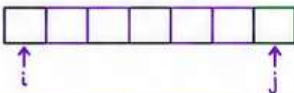


★ 4. Two Pointer Pattern

Use two pointers (`i, j`) moving towards each other or in same direction.

Useful for:

- Palindrome
- Remove duplicates
- Reverse string
- Partition problems



★ 5. Frequency Array

Use an array to store frequency of characters.

For lowercase (a-z):

`int freq[26] = {0};`

For ASCII:

`int freq[128] = {0};`

Increment: `freq[ch - 'a']++`
or `freq[ch]++`

★ 6. ASCII Tricks

char to int → `int x = ch;`
int to char → `char ch = x;`

Lowercase to Uppercase:

`ch - 'a' + 'A'`

Uppercase to Lowercase:

`ch - 'A' + 'a'`

Check if character is letter:

`isalpha(ch)`

MINI FLOW CHART

★ 7. Interview Tips

- Clarify the problem & constraints.
- Think of brute force → optimize.
- Use right data structure.
- Dry run with examples.
- Handle edge cases.
- Write clean & modular code.

